



UNIWERSYTET VIZJA

Zaawansowane zagadnienia algorytmiki i programowania Projekt

Algorytmy w teorii liczb i kryptografia

**Projekt dotyczy implementacji algorytmów arytmetyki modularnej
i teorii liczb w kontekście kryptografii (RSA)**

Prowadzący

dr hab. inż. Agnieszka Duraj

Studenci

Jakub Peciak Album nr: 65848

Paweł Cabaj Album nr 65456

Warszawa 2026

Raport 4

Analiza i bezpieczeństwo: testy, demonstracja
szyfrowania i dekryptowania, wnioski.

Spis treści

1. Wstępne elementy analizy
2. Komunikacja między konsolami
 - 2.1. Konsola klienta
 - 2.2. Konsola serwera
 - 2.3. Modyfikacja kodu
 - 2.4. Uwagi
3. Diagram komunikacji
4. Pomiary
5. Prezentacja wyników
 - 5.1. Oczekiwanie klienta
 - 5.2. Procedowanie serwera
 - 5.3. Czas na komunikację
 - 5.4. Czas procesora
 - 5.5. Procentowe użycie CPU

1. Wstępne elementy analizy

Celem tego raportu jest analiza poprawności działania algorytmu według tematu programowego – projektowego, dotyczącego mechanizmu z zakresu kryptografii - jakim jest generacja klucza RSA. W raporcie tym przedstawiono badanie i opracowanie przede wszystkim czasu generacji klucza oraz czasu oczekiwania na odpowiedź przez klienta.

Projekt jest przygotowany w modelu klient - serwer i tak jest realizowana. Tutaj skupiamy się na modelu, który reprezentowany jest przez diagram komunikacji/przepływów (patrz rys). Warto też wspomnieć, iż zagadnienie RSA jest tematem, który otwiera szeroką przestrzeń dotyczącą szyfrowanej komunikacji i jest to powszechnie stosowana technologia.

W skrócie mówiąc chodzi o relacje żądanie - odpowiedź, która jest realizowana programami projektowymi. Jakkolwiek efektem jest publiczny klucz po stronie klienta oraz klucz prywatny po stronie Serwera. Klient może przysyłać szyfrowane komunikaty oparte o ten właśnie klucz, jaki otrzymał. Taki scenariusz gwarantuje odczytanie informacji klienta kluczem prywatnym po stronie serwera, który to serwer sam wygenerował tę parę kluczy.

Zagadnienie RSA jest algorytmem z dziedziny kryptografii asymetrycznej, bo taki jest właśnie kontekst jego użycia. Aby zapewnić kompleksowe rozwiązanie do w pełni szyfrowanej, szybkiej i bezpiecznej komunikacji, należałoby i tak się w praktyce stosuje - metody hybrydowe. Polegają one na wykorzystaniu RSA, jako inicjatora procesu do stworzenia logicznie bezpiecznego kanału przesyłanie komunikatów.

Algorytm opiera się na kosztownych obliczeniowo operacjach arytmetycznych na dużych liczbach całkowitych. Jakkolwiek jest to protokół praktyczny w użyciu, lecz nie gwarantujący warunków zadowalającego szyfrowania danych. Po pierwsze jest to model asymetryczny (jednostronny) – klucz prywatny „odczyta” informację kodowaną kluczem publicznym a klucz publiczny „nie odczyta” wiadomości prywatnej. Po drugie jest niepraktyczny z punktu widzenia złożoności arytmetycznej, konkurencyjne rozwiązania są wydajniejsze. Po trzecie długość wiadomości szyfrowanej zależy od długości klucza i dla przykładu, dla klucza 2048-bit jest to rząd kilkuset bajtów. I po trzecie, złamana atakiem komunikacja jest całkowicie do odtworzenia.

2. Komunikacja między konsolami

Ponieważ programy nie mają powłoki graficznej GUI ani własnej listy poleceń kompilacyjnych, program klienta i serwera trzeba uruchomić z poziomu narzędzia IDEA. Tu użyto VSCode. Programy działają w gruncie rzeczy synchronicznie, jednak zostaje zachowana konwencja żądanie – odpowiedź. Od strony praktycznej najpierw należy uruchomić serwer, aby uniknąć błędów później zaś klienta.

2.1. Konsola klienta

- Serwer musi być już uruchomiony – jeśli rzeczywiście ma to być usługa
- Klient uruchamia konsolę i komendą „rsa” inicjuje proces generacji
- Wpisuje długość klucza i wysyła żądanie.
- Oczekuje na odpowiedź

Tu działa serwer →

- Po obsłużeniu żądania dostaje z serwera klucz publiczny - parę (n, e)
- Podaje parametr jak długo oczekiwał oraz użycie czasu procesora i bloków pamięci
- Aby wygenerować klucz ponownie nie trzeba opuszczać czy restartować konsolę. Wystarczy ponowić „rsa” i długość klucza.

```
Z SERWERA - wiadomość nieszyfrowana: Połączenie otwarte..  
Napisz wiadomość (lub 'exit' aby zakończyć): rsa  
Długość klucza: 48
```

```
🔑 Otrzymano klucz publiczny RSA:  
n= 180124762923031  
e = 77105296862051
```

```
Czas oczekiwania na klucz: 0.0019425998907536268  
Zużycie CPU (czas użytkownika): 0.000000 s  
Przyrost pamięci: 4096 B  
Napisz wiadomość (lub 'exit' aby zakończyć): []
```

2.2. Konsola serwera

- Serwer po uruchomieniu nasłuchuje na gnieździe IP 127.0.0.1 + port 8888.

Tu klient zgłasza żądanie →

- Serwer dostaje zgłoszenie klienta
- Wylicza klucze
- Wypisuje potwierdzenie długości klucza oraz zmierzony czas procedowania i użycie czasu procesora.
- Potwierdza przesłanie

```
Rozpaczynam nasłuchiwanie na : 127.0.0.1:8888
DO KLIENTA - wiadomość nieszyfrowana: Połączenie otwarte..

[OD KLIENTA] Potwierdzam
Napisz wiadomość (lub 'exit' aby zakończyć):
[OD KLIENTA] GENERUJ_RSA 48
Żądanie od klienta: wygeneruj klucz RSA.

Dokładna długość klucza: 48 bit

Czas generacji klucza: 0.0008193999528884888
Zużycie CPU (czas użytkownika): 0.000000 s
Przyrost pamięci: 0 B
Wysłano klucz publiczny (n,e) do klienta.
█
```

2.3. Modyfikacja kodu

W treści algorytmu serwera dokonano jeszcze jednego usprawnienia. Przygotowano drugą wersję funkcji do generowania klucza, która przyjmuje jako parametr długość jaką ma mieć klucz. To bardzo ułatwiło generowanie logów do potrzeb testów - takich jak zużycie pamięci, czasu procesora czy czasu generacji kluczy.

Funkcja ma w sobie dodatkową pętlę while, którą powtarza aż do momentu osiągnięcia właściwego rozmiaru. Ponieważ ($n = p * q$) długość klucza będzie oscylowała wokół wartości sumy długości (p i q). Ponieważ funkcja `< generate_prime( ) >` przyjmuje ($\min$ i $\max$) inicjujemy wartością $\frac{1}{2}$ docelowej długości klucza.

Pętla daje znikomą narzut na czas wykonania generacji, a znacząco ułatwia przygotowanie zestawów testowych.

```
# Algorytm RSA - określona długość
def generate_rsa_fixed(x):

    global global_n, global_d

    y = x // 2
    min, max = n_bit(y)

    while True:
        p = generate_prime(min, max, None)
        q = generate_prime(min, max, p)
        n = p * q
        if n.bit_length() == x:
            break # mamy idealny rozmiar

    phi = (p - 1) * (q - 1)
    e = generate_e(phi)
    d = generate_d(e, phi)
    global_d = d
    global_n = n

    bit_length = n.bit_length()
    print(f"\nDokładna długość klucza: {bit_length} bit")

    return n, e, d
```

Dla ułatwienia przygotowano funkcję `< n_bit( ) >`, która przyjmując jako parametr pożądaną długość klucza, przelicza to na wartości dziesiętne gotowe do podania do funkcji `< generate_prime( ) >`. Funkcję równie dobrze można byłoby umieścić powyżej.

```
def n_bit(n):
    min_val = 2**(n-1)
    max_val = 2**n - 1
    return min_val, max_val
```

Do potrzeb testowych było konieczne zautomatyzowanie w jakimś zakresie zbierania danych testowych. Tu z kolei usprawnienie polegało na fakcie, iż razem z prośbą o wygenerowanie klucza, klient wysyła także długość klucza jaki chce otrzymać – w jednej zmiennej. Jest to zabieg jedno linkowy. Także po stronie serwera umieszczono obsługę komunikatu (RSA + długość). Zmiana była 3 linijkowa. Należało rozdzielić komunikat z powrotem na RSA i długość.

```
parts = msg.split()
cmd = parts[0]
global_len = parts[1] if len(parts) > 1 else None
```

2.4. Uwagi

Aplikacja testowa nie działała w sieci lokalnej a wyłącznie na localhost. Czas pracy procesora po stronie klienta jest niemierzalnie mały. Z kolei dla serwera pomijalnie małe jest zużycie zasobów pamięci.

W pomiarach nie wzięto pod uwagę scenariusza z generacją liczb pierwszych bez testu Rabbina – Millera (nazwijmy to metodą legacy). Czasy generacji nawet dla małych kluczy były zdecydowanie długie i bez praktycznego zastosowania. Funkcja jednak została w projekcie jako nie aktywna. W każdym momencie można ją testować.

3. Diagram komunikacji

Poniżej opisu znajduje się powstały na podstawie toku przygotowania projektu schemat logiczny interakcji klient – serwer, uwzględniający naszą implementację oraz potencjalny kierunek ewaluacji projektu.

Co do dyskusji samego diagramu... Opisuje on w sposób graficzny interakcje jakie zachodzą pomiędzy serwerem a klientem. Przede wszystkim serwer czeka na jakąkolwiek aktywność nasłuchując na określonym IP i określonym porcie. U nas jest to localhost, 127.0.0.1 a port to 8888 . W ten sposób oczekuje na żądania, która uruchamiają jakąś logikę na serwerze. W naszym przypadku jest to generacja klucza RSA. W oddzielnej instancji uruchomiony zostaje klient. Wysyła komunikat z żądaniem podjęcia jakiegoś działania - u nas jest to generowanie klucza.

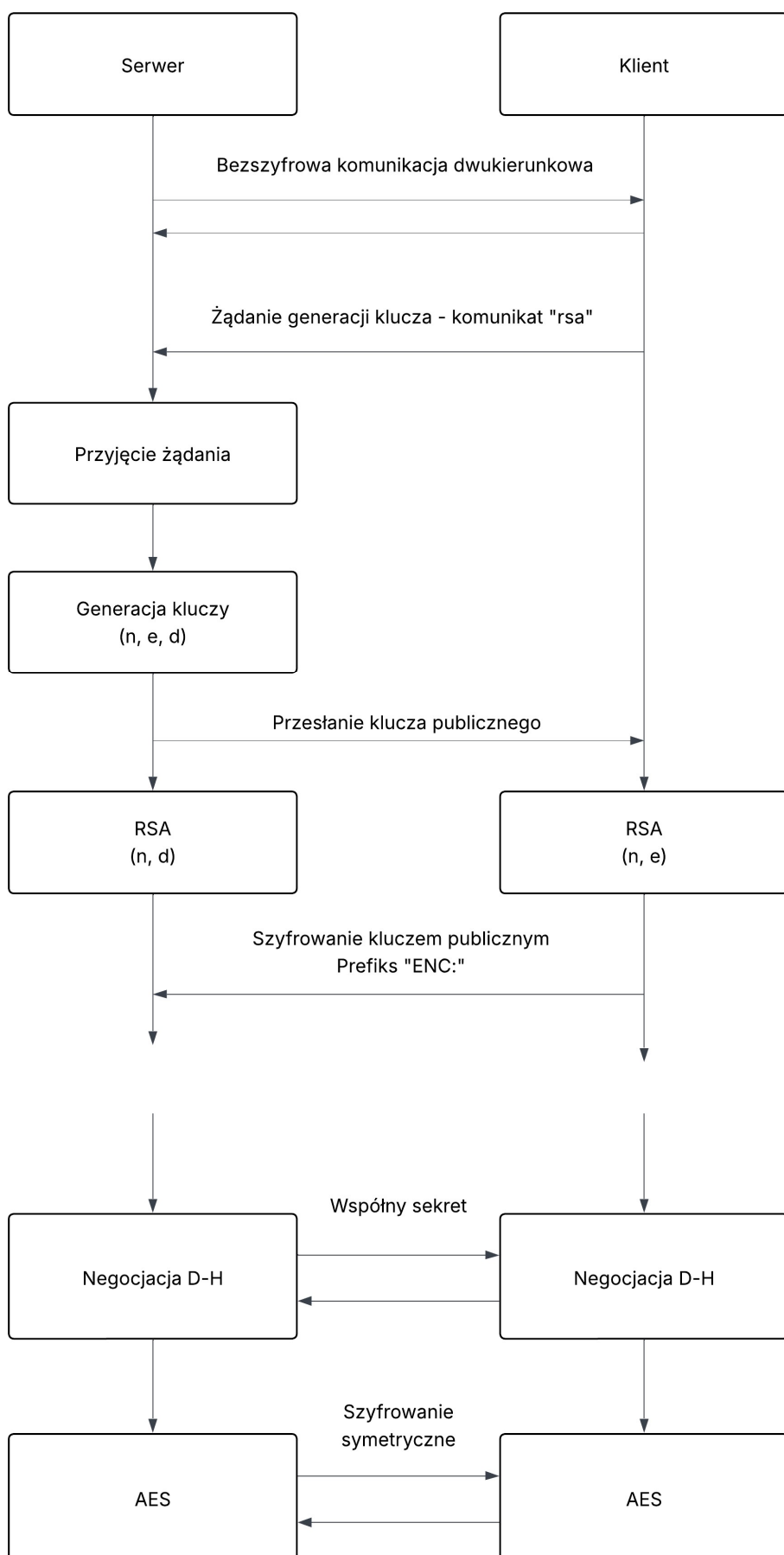
Programy serwer i klient zostały napisane w ten sposób, że możliwa jest bez szyfrowa komunikacja dwukierunkowa, którą wykorzystamy do przestania komunikatu z prośbą o wygenerowanie pary kluczy - publicznego i prywatnego (obu). Komunikaty można wymieniać na konsolach. Jednak wpisując **komunikat „rsa”** klient uruchamia żądanie. Serwer identyfikując ruch dostaje dyspozycję, aby przygotować i wygenerować klucze. Po generacji serwer zachowuje dla siebie parę (n, d) - jako prywatną, dotyczącą danego klienta a klientowi odsyła klucz publiczny, który służy do przygotowywania szyfrowanych komunikatów możliwych do odczytania tylko po stronie serwera.

Od tej pory w naszym przykładzie można wysyłać komunikaty szyfrowane wpisując na konsoli tekst poprzedzony **prefiksem „ENC:”**.

Teraz jesteśmy w stanie utworzyć symetrycznie szyfrowane połączenie korzystając z innych technologii kryptograficznych, które nie są przedmiotem projektu, jednak uzupełniają informacje o tym, jaka jest rola RSA w obecnej kryptografii.

Powszechnie używa się negocjacji Diffiego – Hellmana, która polega z grubsza na wyznaczeniu tajnego sekretu znanego tylko obu stronom w komunikacji, na podstawie wymiany liczb i algorytmami potęgowego. Tak utworzone znane dwóm stronom hasło może być po przekształceniach podstawą do wyznaczenia i użycia klucza do algorytmu symetrycznego szyfrowania AES.

Sama rola RSA ogranicza się do uwierzytelnienia stron w komunikacji. Stąd na diagramie przerwa w wątku przepływu symbolizująca fakt rozpoczęcia bezpiecznej komunikacji szyfrowanej. Wymagającej oczywiście uzgodnień, o których wspomniano. Jest jednak bazą do wyznaczenia parametrów tychże protokołów.



4. Pomiar

Sposób dokonania pomiarów od strony kodu był opisany w poprzednim raporcie. Pomiarów dokonano dla opisanej w tym raporcie konfiguracji. Na bazie załączonych plików z kodem testowym.

Długość klucza [b]	Czas oczekiwania klienta [s]	Czas procedowania serwera [s]	Wykorzystania procesora - serwer [s]	Różnica czasów [ms]	Użycie pamięci – klient [B]
48	0.0019426	0.0008194	-		
	0.001666	0.0008194	-		
	0.002095	0.0008194	-		
	0.0021245	0.0008194	-		
Średnia	0.001957025	0.0008194	-	1.1376	4096
124	0.0039198	0.0025082	-		
	0.0032603	0.0019502	-		
	0.0040378	0.0018752	-		
Średnia	0.0037393	0.0021112	-	1.6281	4096
512	0.0146679	0.0136774	0		
			0.03125		
	0.0653165	0.0643632			
	0.029107	0.0269436	0		
Średnia	0.0363638	0.034994733	0.01041	1.3690	8192
1024	0.5403297	0.5385453	0.23437		
	0.609195	0.6074738	0.21875		
	0.6160929	0.6150194	0.28125		
Średnia	0.5885392	0.587012833	0.24479	1.5263	8192
2048	3.637223	3.6358574	1.76562		
	4.9112993	4.9100993	2.25		
	1.317322	1.3161238	0.59375		
	1.7616653	1.7603438	1.35937		
Średnia	2.9068774	2.905606075	1.49218	1.2713	16 384
3072	34.8719857	34.8706005	26.1562		
	12.6609787	12.659361	10.1250		
	54.8515181	54.8499999	43.1093		
Średnia	34.12816083	34.1266538	26.4635	1.5070	65 536
4096	76.7525398	76.7510196	58.3125	1.5202	

5. Dyskusja wyników

Patrząc na zestawienie tabelaryczne widać, że parametry czasowe i dla serwera i dla klienta w gruncie rzeczy odpowiadają sobie, co oznacza, że czas oczekiwania klienta jest w przybliżeniu równy czasowi procedowania serwera. Narzut wynosi w przybliżeniu ~ 1.5 ms bez względu na długość bitową klucza. Potwierdza to, że dominuje długość czasu obliczeń kryptograficznych. A ich złożoność jest krytyczna.

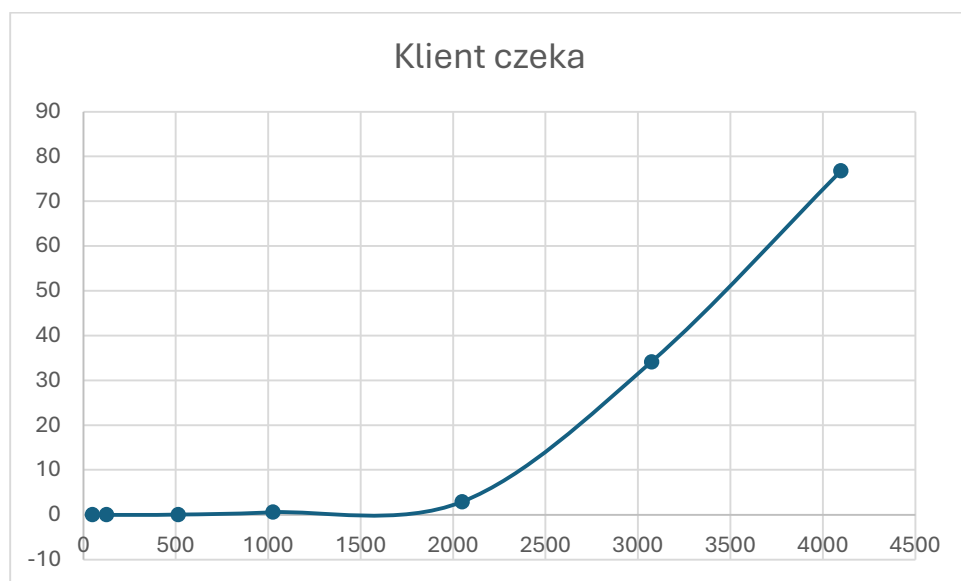
Czas oczekiwania klienta i odpowiedzi serwera gwałtownie rosną w trakcie zmieniania zadanej długości klucza. O ile dla klucza 1024 bit obliczenia trwają ~ 0.5 s, o tyle dla 4096 bit zdecydowanie ponad 1 minutę. Wzrost ten ma charakter potęgowy.

Użycie procesora po stronie serwera, gdzie widać, że algorytm mocno obciąża wątek procesora, potwierdza słabą skalowalność rozwiązania oraz ryzyko atakami typu DoS. Zużycie procesora po stronie klienta jest nie rejestrowalnie małe.

Podobnie małe jest też zużycie pamięci po stronie serwera, natomiast po stronie klienta rośnie wraz z długością klucza i dla dużych wartości zaczyna być może nie krytyczna, ale zauważalna - na poziomie 65 kB.

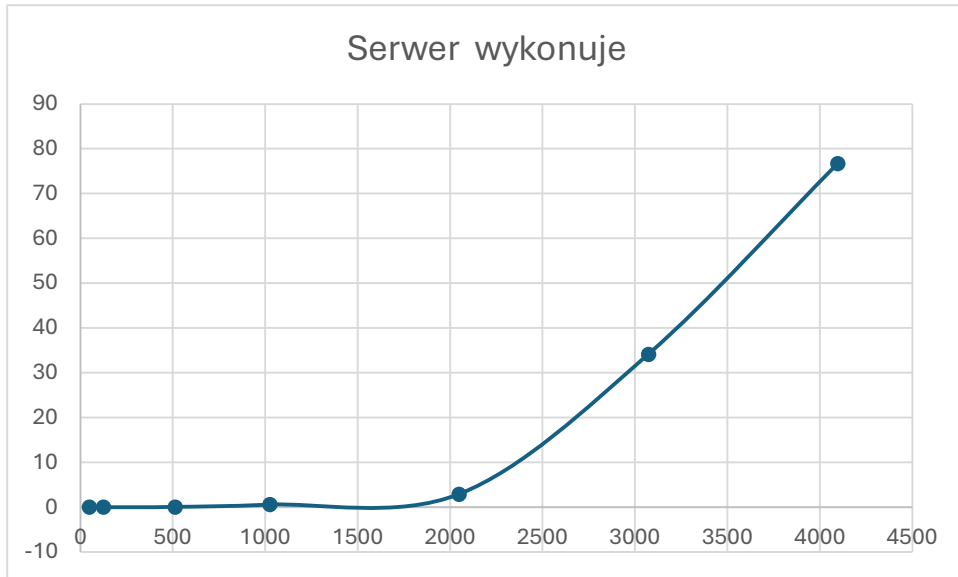
5.1. Oczekiwanie klienta

Długość klucza [b] – czas [s]



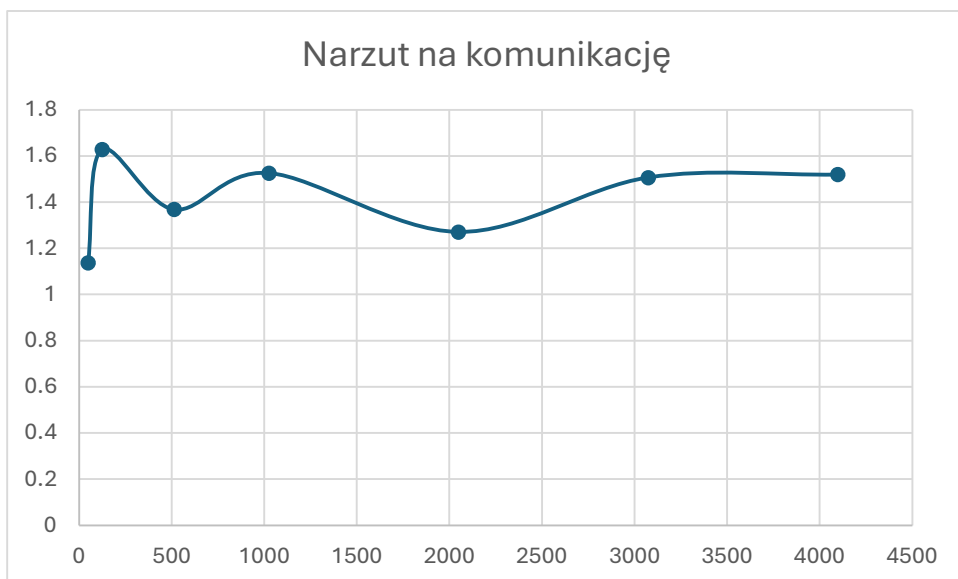
5.2. Procedowanie serwera

Długość klucza [b] – czas [s]



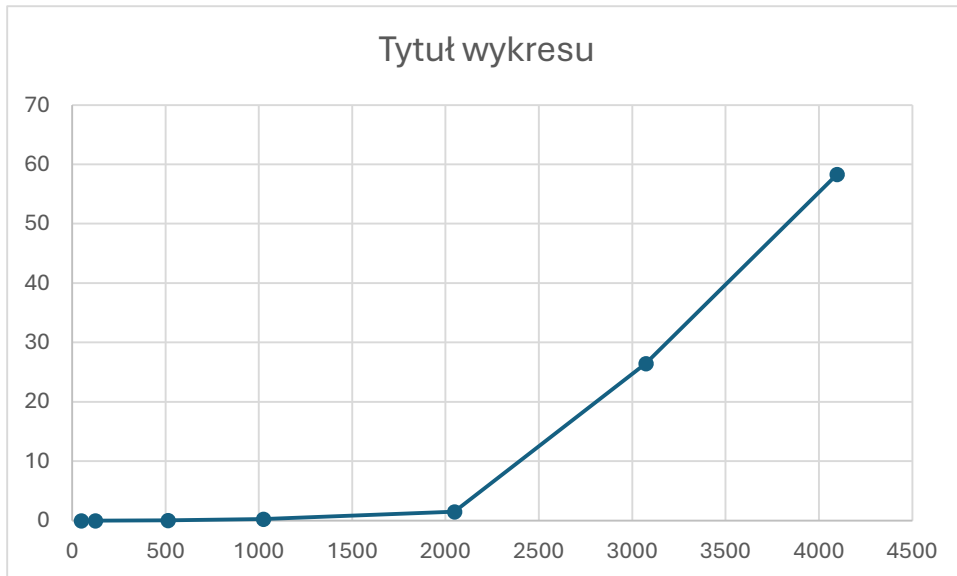
5.3. Czas na komunikację

Długość klucza [b] – czas [ms]



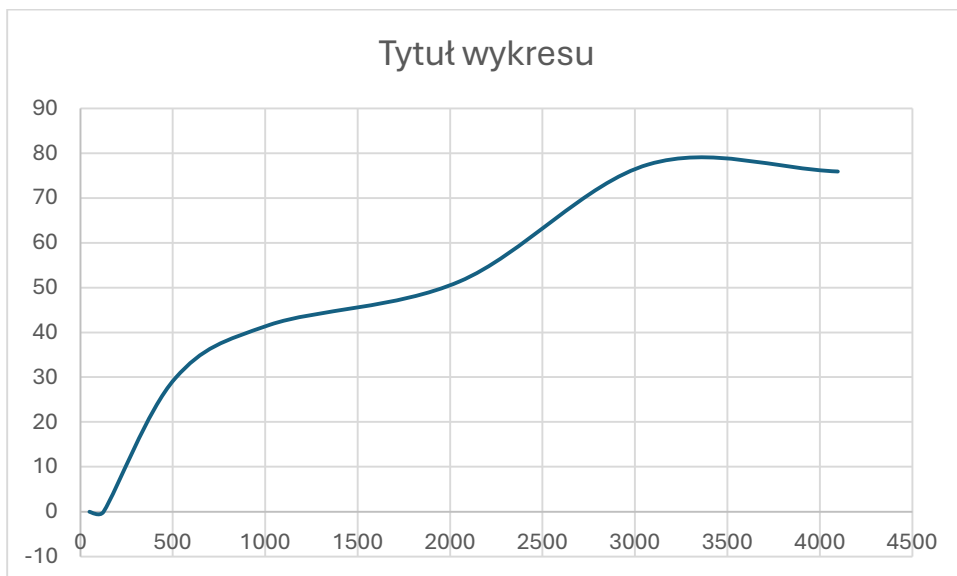
5.4. Czas procesora

Długość klucza [b] – czas [ms]



5.5. Procentowe użycie procesora

Długość klucza [b] – udział CPU [%]



Procentowe użycie procesora jest tu podane w odniesieniu do czasu generacji klucza RSA. Określa ono jaki procent czasu generacji zajmuje aktywność procesora. Widać, że rośnie on wraz ze wzrostem długości bitowego klucza, ale prawdopodobnie ustala się na poziomie bliskim lub równym 80%. Sam sposób obliczania tego parametru podano poniżej.

$$T_{CPU} = \frac{\text{czas procesora}}{\text{czas generacji}} * 100\%$$